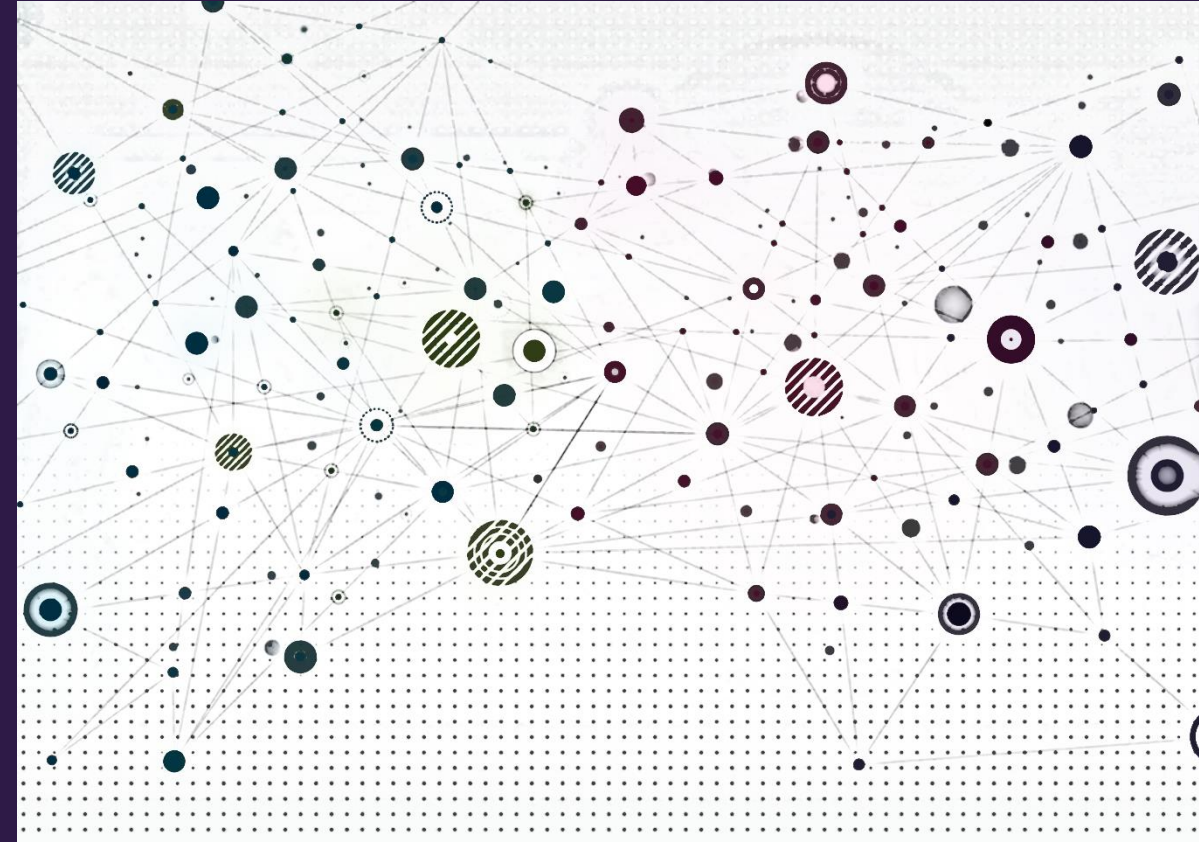


A Generalized Perspective on GNNs





Overview

- Recap
- Message passing framework
- Standard GNNs in the message passing framework
- Applicable deep learning tools
- Stacking GNN layers
- Graph manipulations



Recap





Deep learning on graphs

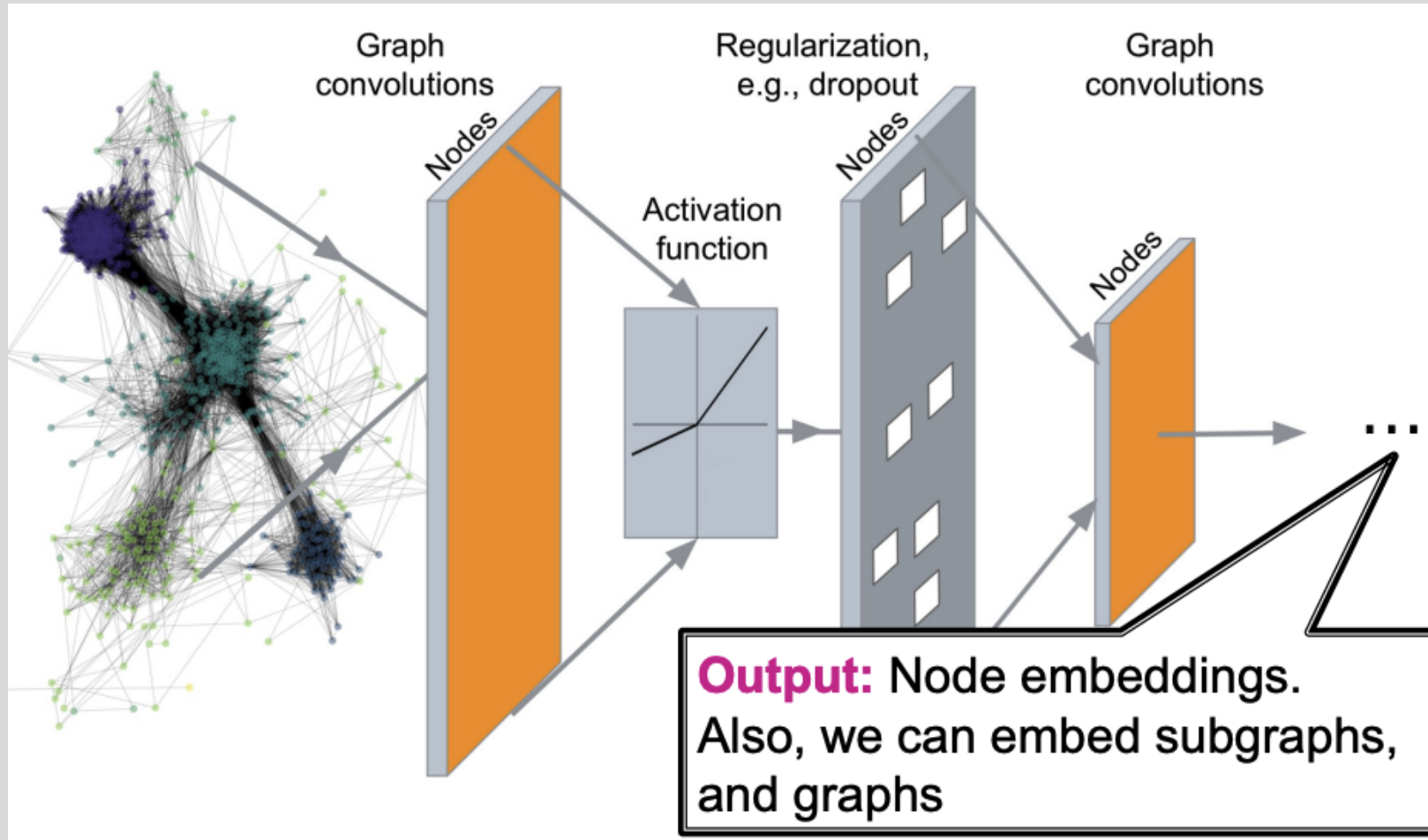


Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



Graph deep learning paradigm

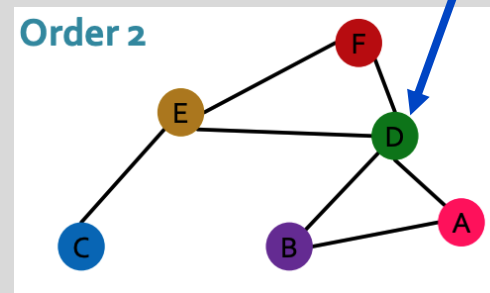
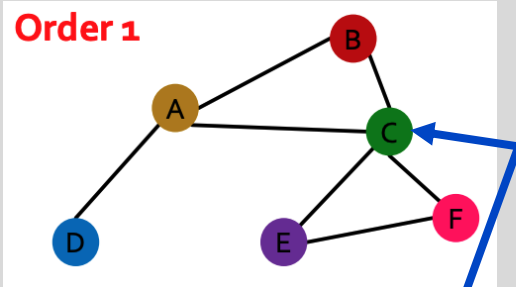
- Assume we have a graph G
- V is the set of vertices in G
- A is the adjacency matrix (assume unweighted, undirected for now)
- $X \in \mathbb{R}^{|V| \times d_l}$ is a matrix of **node features (this is new!)**
 - One d_l -dimensional feature vector per node
 - Examples of node features
 - User profile data, user image
 - Gene expression, gene functional properties
 - What if my nodes don't have features?
 - One-hot embeddings - not recommended as this makes the model transductive
 - Constant vector of 1 – model is again inductive, features will be updated (more to come)



Permutation invariance and equivariance

- Permutation **invariant** –
permute the input, the output is
unchanged

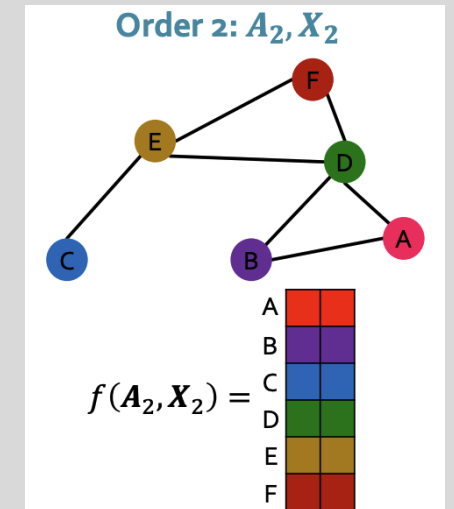
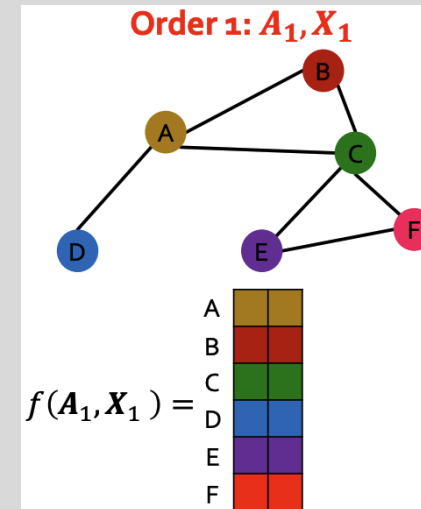
$$f(\mathbf{A}, \mathbf{X}) = f(\mathbf{PAP}^T, \mathbf{PX})$$



Permutation
invariance
holds for node
embeddings
(can also hold
for graph
embeddings
by aggregating
over all node
embeddings)

- Permutation **equivariance** –
permute the input, the output is
permuted in the same way

$$\mathbf{P}f(\mathbf{A}, \mathbf{X}) = f(\mathbf{PAP}^T, \mathbf{PX})$$



Permutation equivariance is property of the
output embedding matrix



GNNs chain permutation invariant and equivariant operations

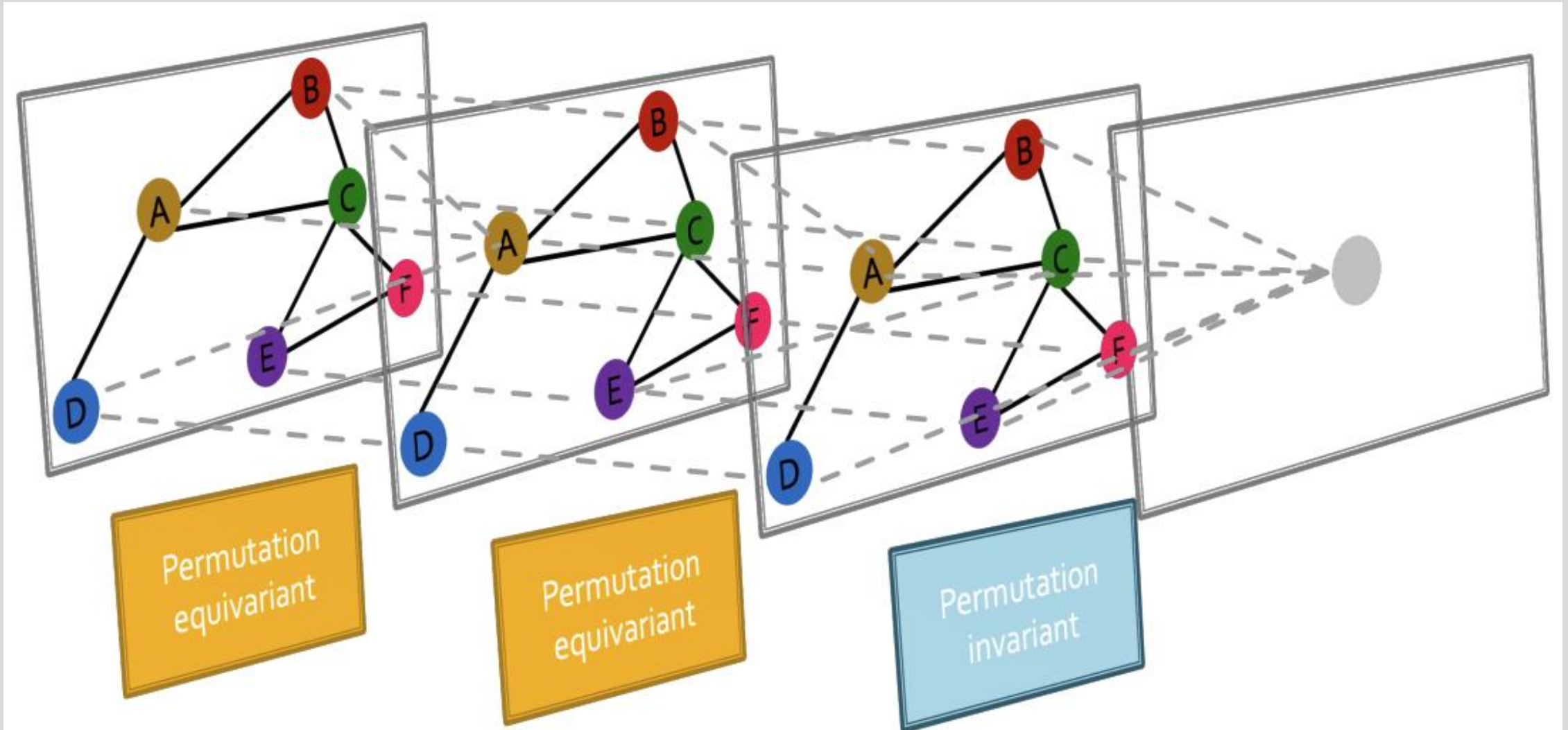


Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



Neighbor node aggregation

- Every neighborhood defines a computation graph

Each node has a unique computation graph based on its neighborhood

Input graph

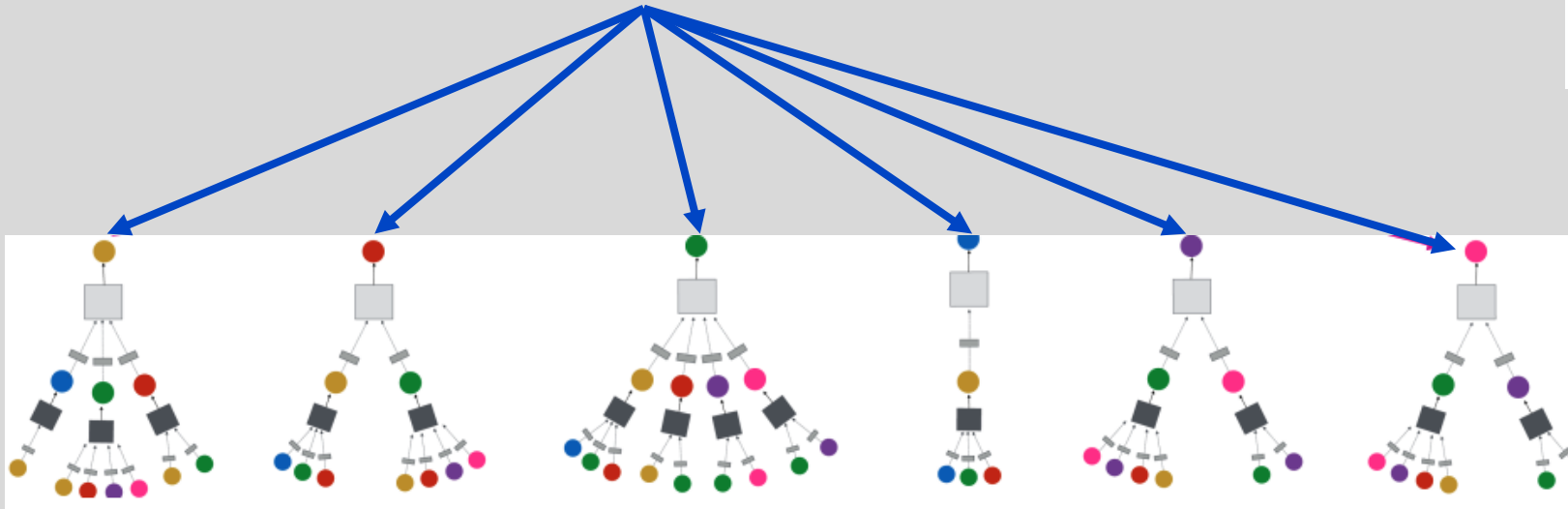
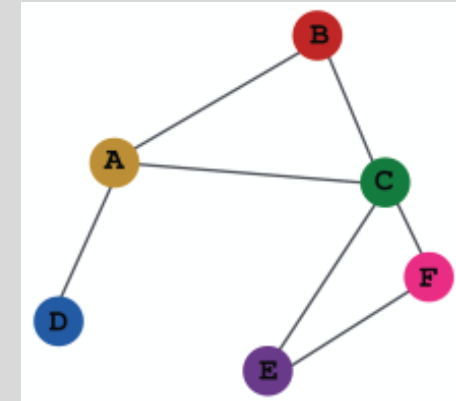


Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



The math of graph convolution networks in matrix form

- From the previous result, we can rewrite the update function over all nodes as:

$$\mathbf{H}^{(k+1)} = \sigma \left(\boxed{\tilde{\mathbf{A}} \mathbf{H}^{(k)}} \mathbf{W}^{(k)T} \right)$$

where $\tilde{\mathbf{A}} = \boxed{\mathbf{D}^{-1}} \mathbf{A}$

normalization

neighborhood
aggregation

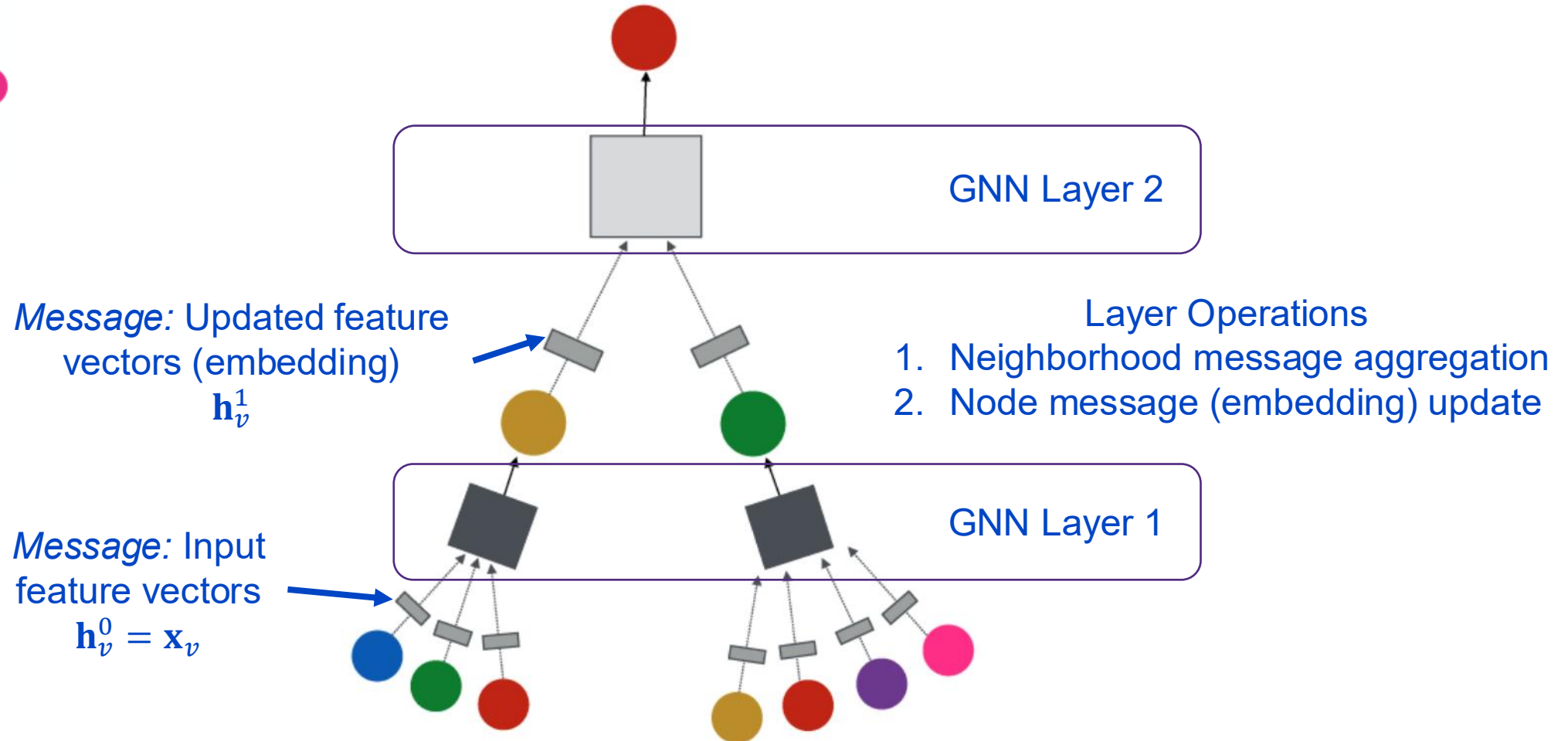
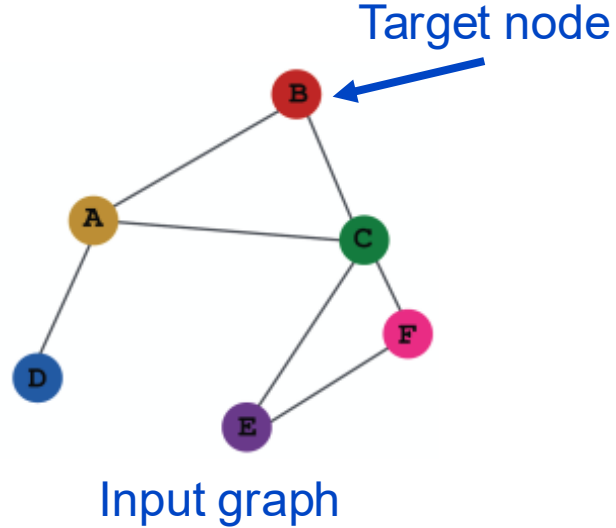
Can we generalize this approach?



Message passing framework



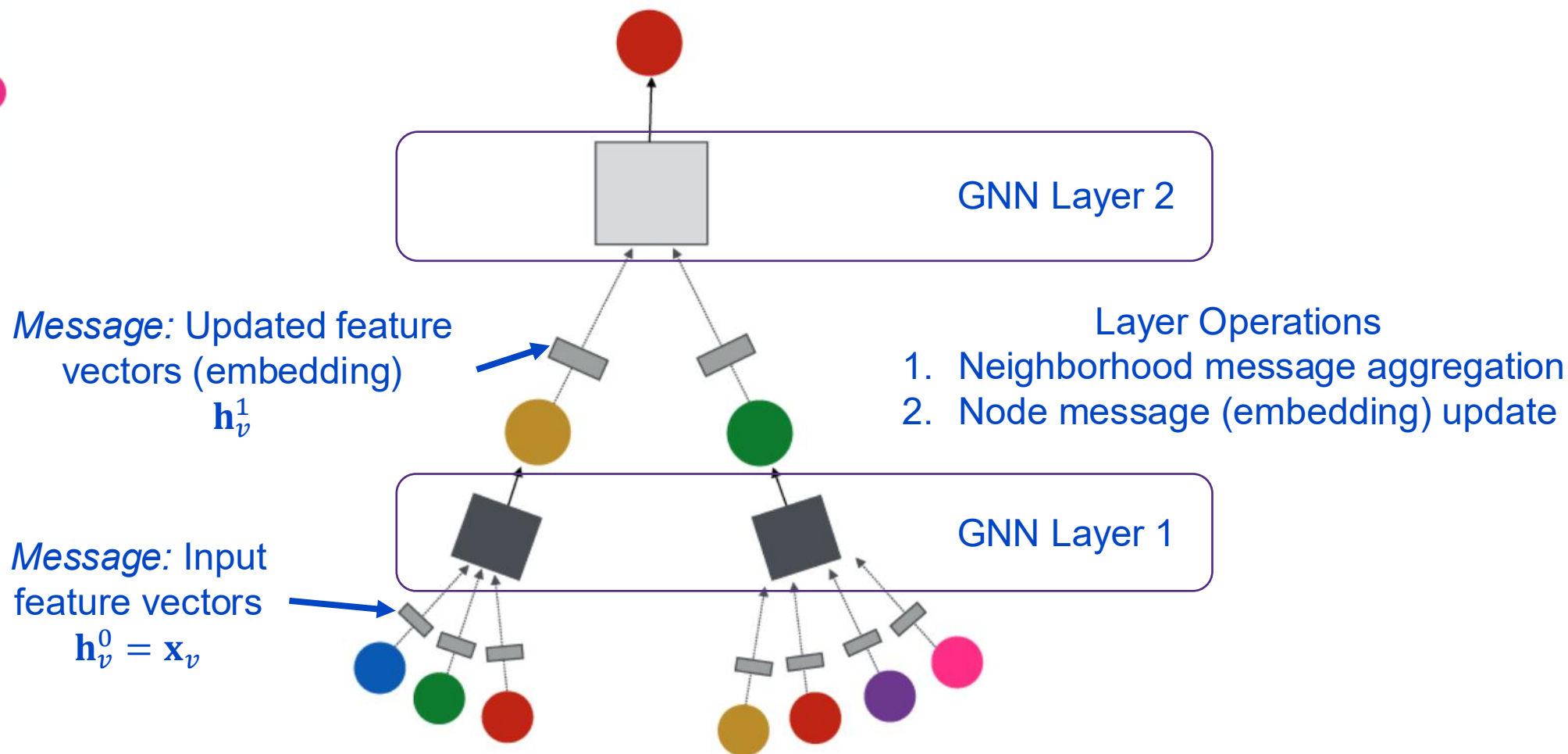
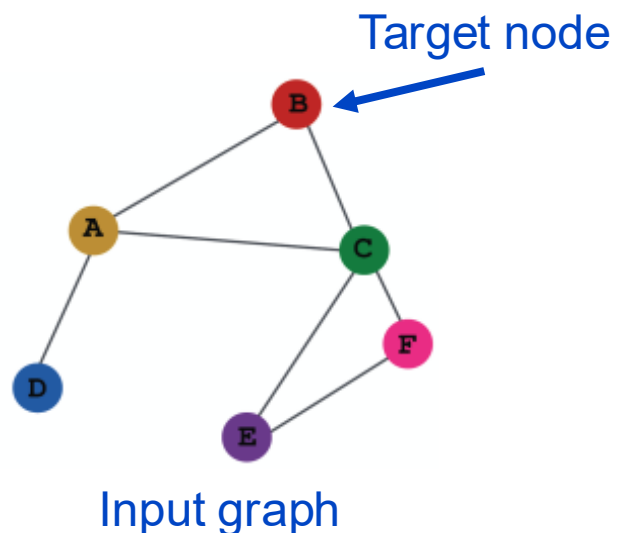
Message passing in GNNs



Message passing in GNNs

GNN Layer $\equiv$ Aggregation $\rightarrow$ Update

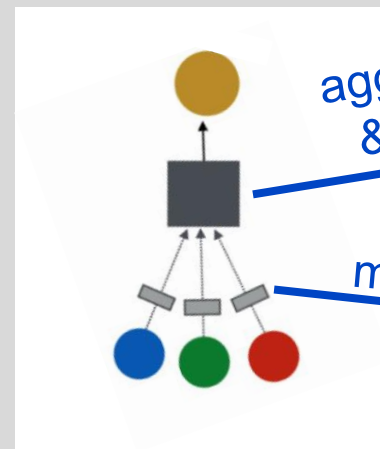
- Different architectures under this perspective
- GCN, GraphSage, GAT, ...



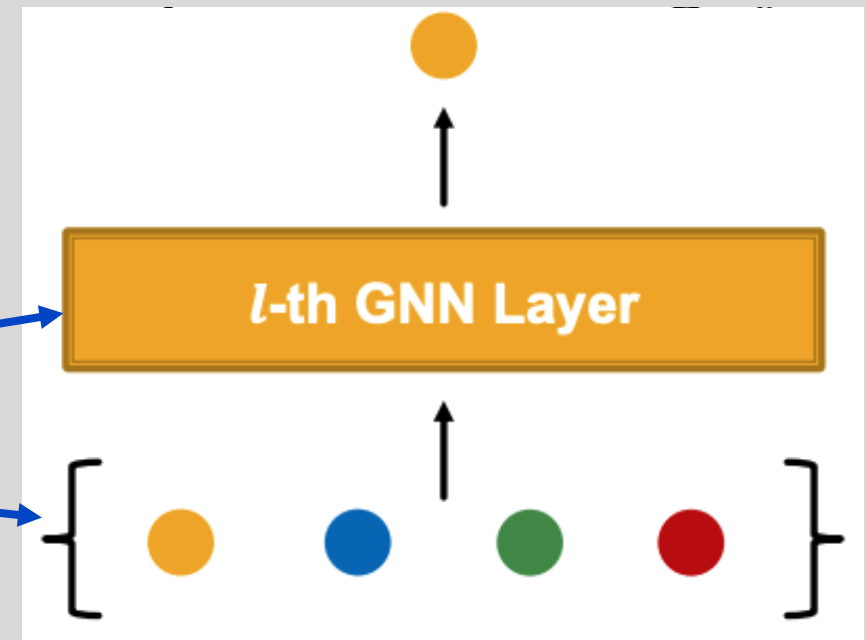


GNN layer concept

- A single GNN layer
- **Main idea:** Convert a set of input vectors into a single vector while maintaining permutation invariance (individual node embeddings) and equivariance (output matrix of all node embeddings)
- **Two step process:**
 1. Node message aggregation
 2. Node embedding update



Updated node embedding $\mathbf{h}_v^l$
(node message input to next layer)



Input node embeddings $\mathbf{h}_v^{l-1}, \mathbf{h}_{u \in N(v)}^{l-1}$
(node's own embedding + neighbors)



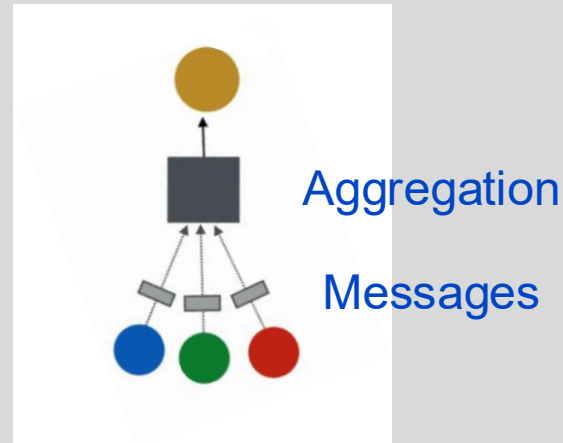
Node neighborhood message aggregation

- Node v will aggregate messages from its neighbors $u \in N(v)$ to form a single neighborhood message:

$$\mathbf{m}_{N(v)}^{(k)} = \text{Agg}^{(k)} \left(\left\{ \mathbf{h}_u^{(k)}, \forall u \in N(v) \right\} \right)$$

where $\text{Agg}^{(k)}$ is any differentiable, permutation invariant function

- Examples: Sum, mean, max (element wise operations)





Node message (embedding) update

- After neighborhood aggregation, node v will **update** its embedding:

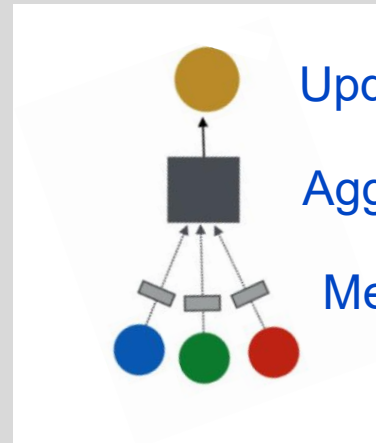
$$\mathbf{h}_v^{(k+1)} = \text{Update}^{(k)} \left(\mathbf{h}_v^{(k)}, \mathbf{m}_{N(v)}^{(k)} \right)$$

where $\text{Update}^{(k)}$ is any **differentiable** function (typically a neural network)

- Example: fully connected linear layer $\mathbf{h}_v^{(k+1)} = \left[\mathbf{h}_v^{(k)} \mid \mathbf{m}_{N(v)}^{(k)} \right] \mathbf{W}^{(k)T}$

update matrix
(linear layer)

vector concatenation



Updated message

Aggregation

Messages



Message aggregation and update as a GNN layer

- Combine the neighborhood message aggregation and node embedding update to form a single GNN layer

1. **Aggregation**: each node aggregates messages (embeddings) of neighbor nodes

$$\mathbf{m}_{N(v)}^{(k)} = \text{Agg}^{(k)} \left(\left\{ \mathbf{h}_u^{(k)}, \forall u \in N(v) \right\} \right)$$

2. **Update**: each node updates its embedding (message for next layer)

$$\mathbf{h}_v^{(k+1)} = \text{Update}^{(k)} \left(\mathbf{h}_v^{(k)}, \mathbf{m}_{N(v)}^{(k)} \right)$$

- The update usually includes a **nonlinearity (activation)**, $\sigma(\cdot)$, such as ReLU, sigmoid



Nonlinear activation function review

- As with standard deep-learning methods, nonlinear activation functions are applied element wise to update a node's embedding
- Sigmoid** – can be useful to restrict range of embedding values, though not the typical choice

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

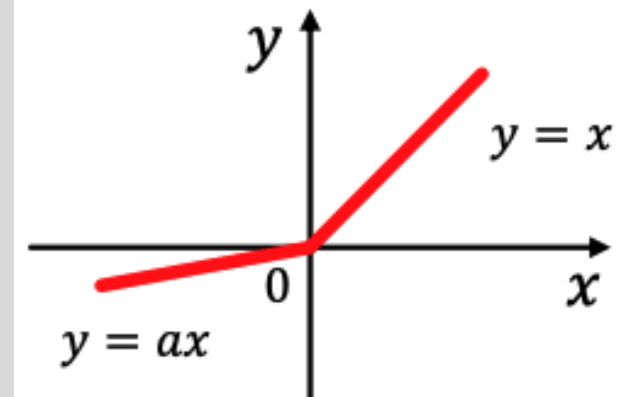
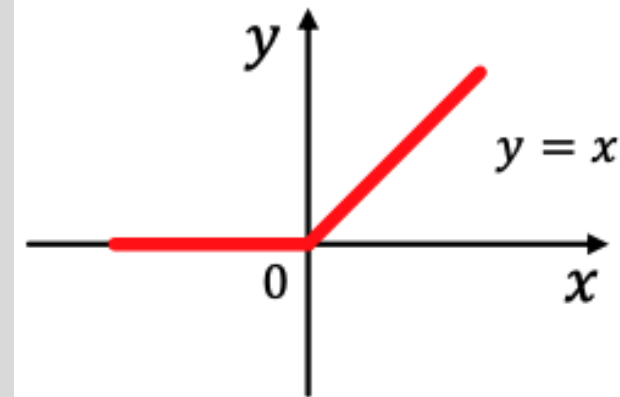
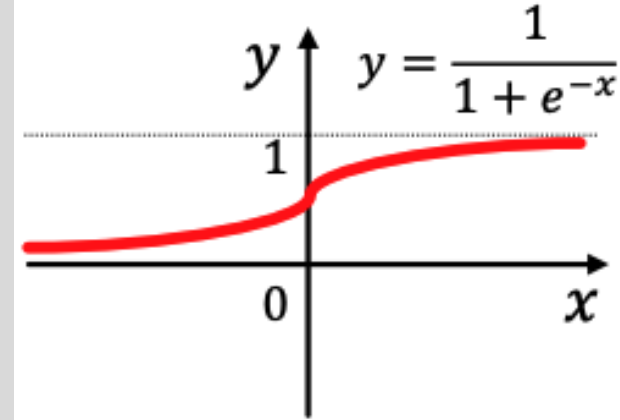
- ReLU** – typical default

$$\text{ReLU}(z) = \max(z, 0)$$

- Parametric ReLU**

$$\text{PReLU}(z_i | a_i) = \max(z_i, 0) + a_i \min(z_i, 0)$$

- a_i is a trainable parameter
- empirical evidence suggests better performance than ReLU





GNNs in the message passing framework





Graph convolutional networks

- A GCN can be written as

$$\mathbf{h}_v^{(k+1)} = \sigma \left(\mathbf{W}^{(k)} \sum_{u \in N(v) \cup \{v\}} \frac{\mathbf{h}_u^{(k)}}{|N(v)|} \right)$$

Recall: Graph augmented with self loops

- How to write this as aggregation $\rightarrow$ update?

$$\mathbf{m}_{N(v)}^{(k)} = \sum_{u \in N(v) \cup \{v\}} \frac{\mathbf{h}_u^{(k)}}{|N(v)|}$$

$$\mathbf{h}_v^{(k+1)} = \sigma \left(\mathbf{W}^{(k)} \mathbf{m}_{N(v)}^{(k)} \right)$$



Graph convolution networks in matrix form

- The GCN can be written in matrix form as

$$\mathbf{H}^{(k+1)} = \sigma \left(\tilde{\mathbf{A}} \mathbf{H}^{(k)} \mathbf{W}^{(k)T} \right)$$

where $\tilde{\mathbf{A}} = \mathbf{D}^{-1} \mathbf{A}$

- How to write this as aggregation $\rightarrow$ update?

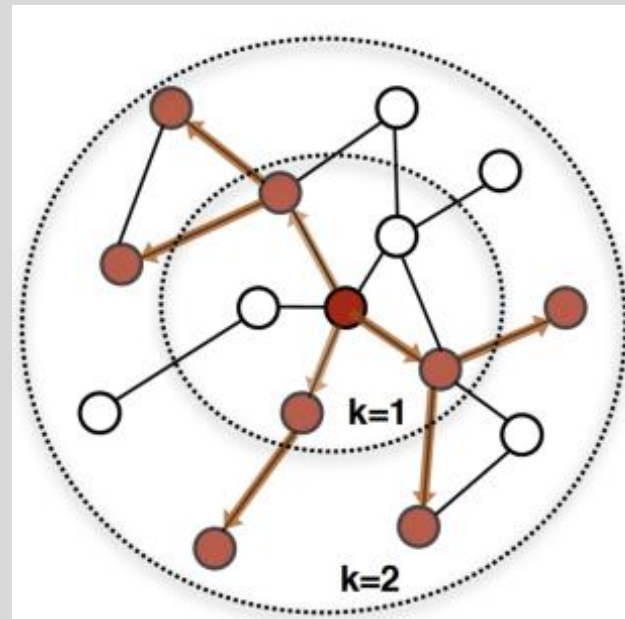
$$\mathbf{M}^{(k)} = \tilde{\mathbf{A}} \mathbf{H}^{(k)}$$

$$\mathbf{H}^{(k+1)} = \sigma \left(\mathbf{M}^{(k)} \mathbf{W}^{(k)T} \right)$$

where $\mathbf{M}^k \in \mathbb{R}^{|V| \times d_k}$ is the matrix of message aggregation function outputs for all nodes (i.e., vector at row i is the aggregated message of the neighbors of node i (from the adjacency matrix ordering))

GraphSAGE (SAmple and aggreGatE) networks

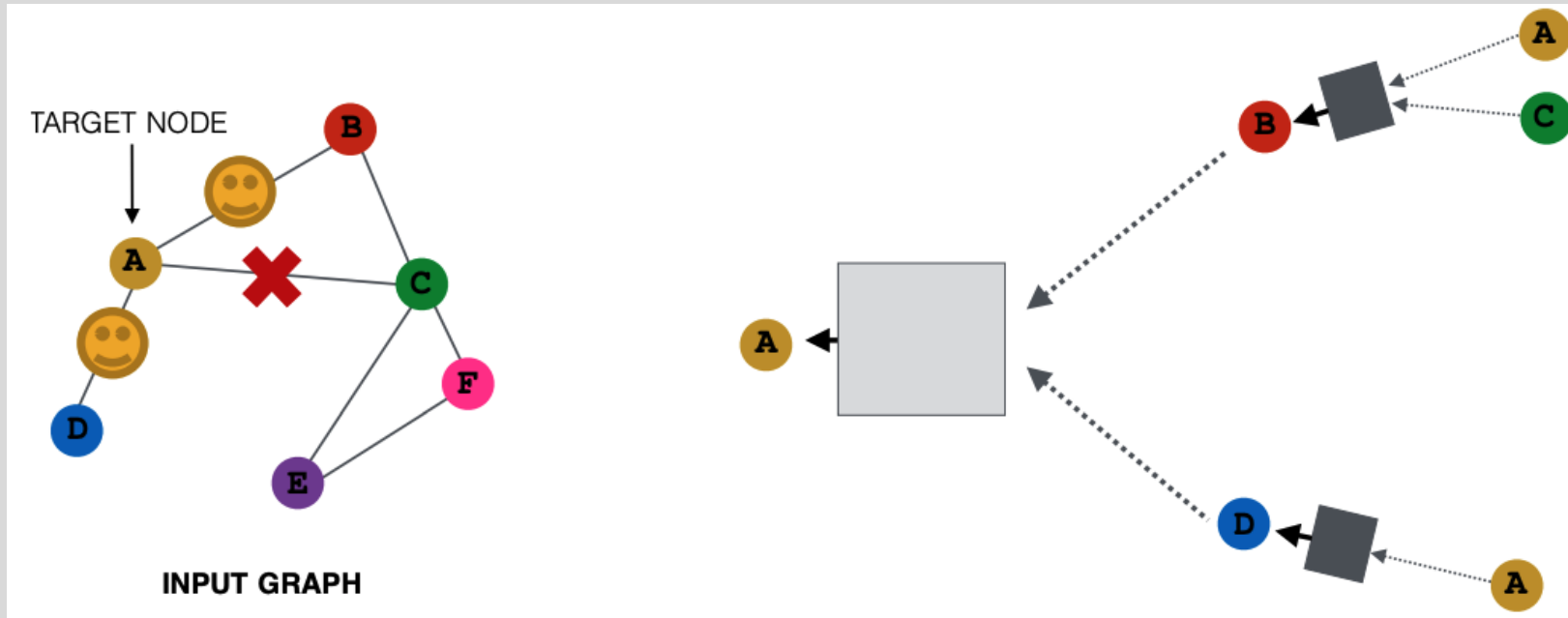
- Message aggregation over all neighbors can be computationally expensive for large / dense graphs or high-degree nodes
- Instead, randomly sample a fixed sized subset of $N(v)$ for every new aggregation





GraphSAGE (SAmple and aggreGatE) sampling

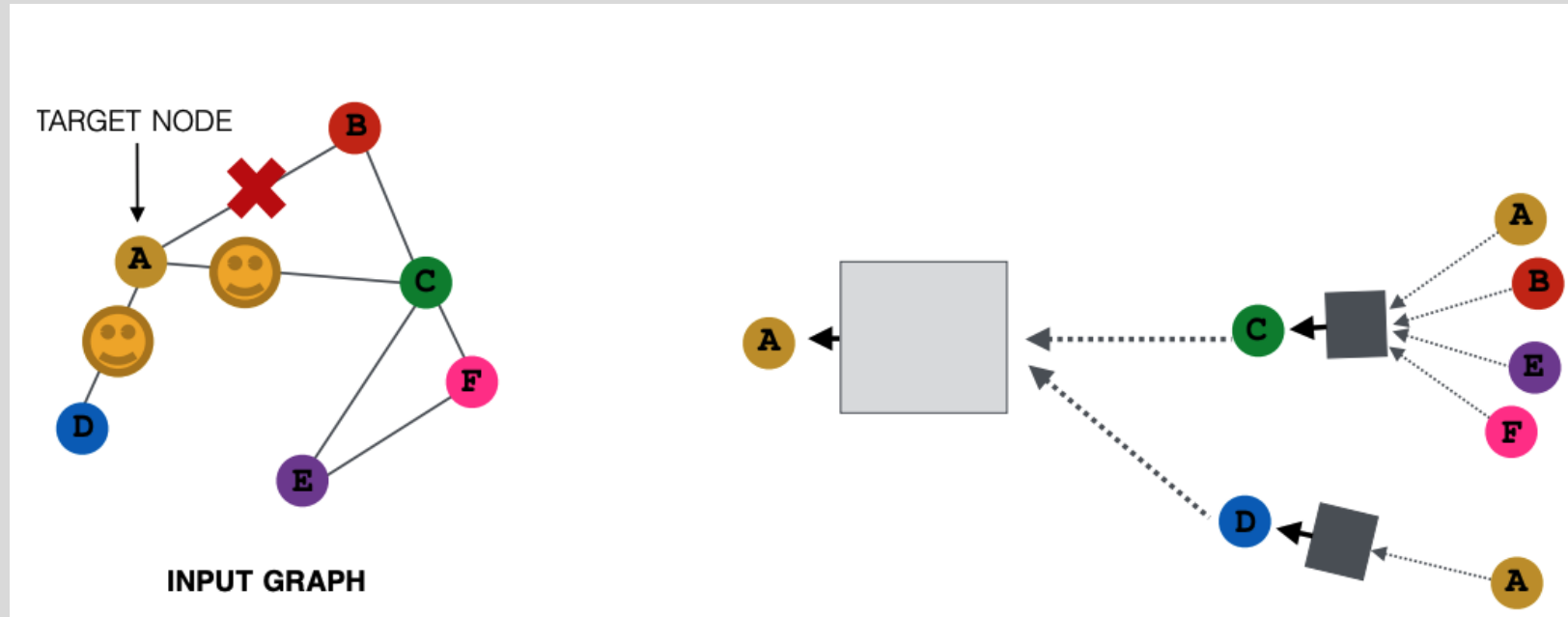
- For example, sample two (2) neighbors for message aggregation
 - Here, only nodes B and D pass messages to A





GraphSAGE (SAmple and aggreGatE) sampling

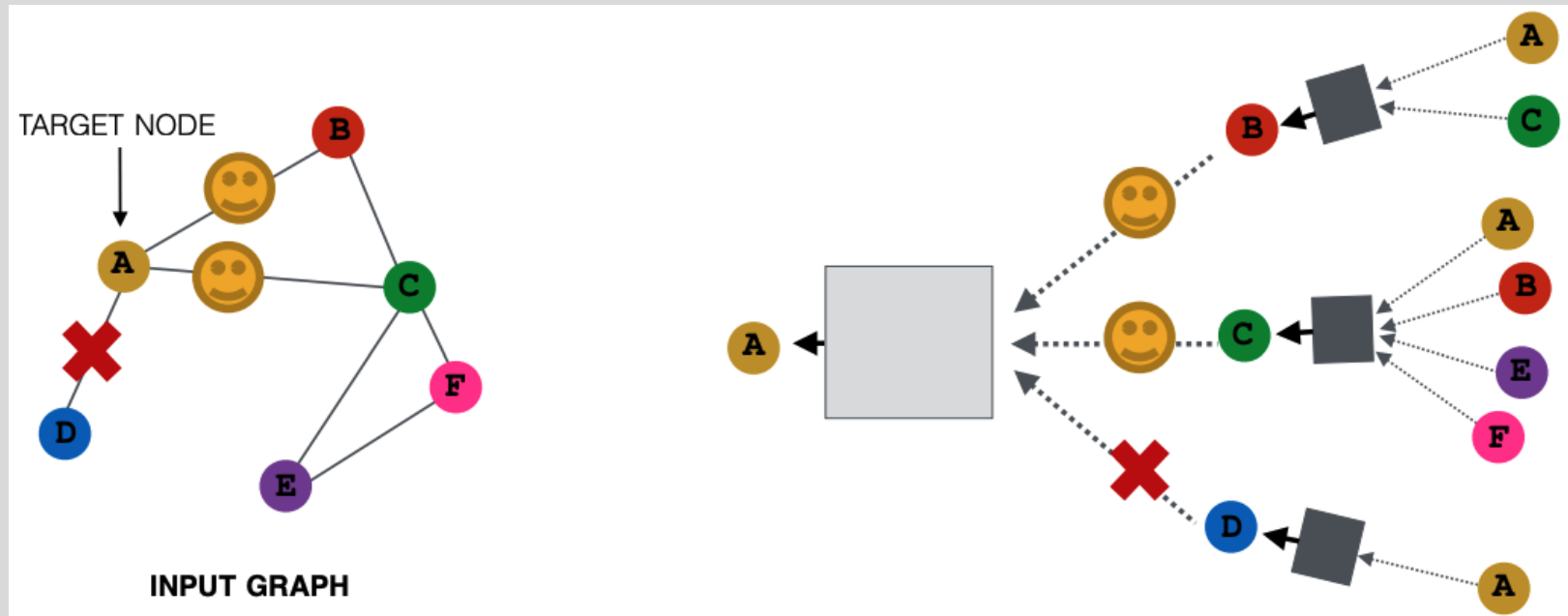
- For the next embedding update (layer, training batch, new sample), we can sample different neighbors
 - Here, only nodes C and D pass messages to A





GraphSAGE (SAmple and aggreGatE) sampling

- In expectation (on average), we get embeddings similar to aggregating over all neighbors
 - Greatly reduces computational cost





GraphSAGE node message aggregation

- Aggregate the messages from the sampled subset of $N(v)$ using permutation invariant function
- Weighted average:** as in GCN

$$\mathbf{m}_{N(v)}^{(k)} = \sum_{u \in N(v)} \frac{\mathbf{h}_u^{(k)}}{|N(v)|}$$

- Pool:** Apply an *MLP* to each node (same MLP on every node) and then apply an element wise vector function, $g(\mathbf{x}) \in \mathbb{R}^d \rightarrow \mathbb{R}^{\tilde{d}}$, such as mean or max to the set of transformed embeddings

$$\mathbf{m}_{N(v)}^{(k)} = g \left[\left\{ MLP \left(\mathbf{h}_u^{(k)} \right), \forall u \in N(v) \right\} \right]$$



GraphSAGE node message aggregation

- **Janossy pooling**: apply a permutation-sensitive function and average over many permutations
- Let $\pi_i \in \Pi$ denote a permutation function that maps the set $\{\mathbf{h}_u, \forall u \in N(v)\}$ to the sequence $(\tilde{\mathbf{h}}_1, \dots, \tilde{\mathbf{h}}_{|N(v)|})$ (here, $N(v)$ is the randomly sampled subset of neighbors)
- Consider a random sampling of permutation orders, Π
- Aggregate node messages as

ρ_Φ is a **permutation-sensitive** function, usually an LSTM

$$\mathbf{m}_{N(v)}^{(k)} = MLP_{\Theta} \left(\frac{1}{|\Pi|} \sum_{\pi \in \Pi} \rho_\Phi [\pi_i(\{\mathbf{h}_u, \forall u \in N(v)\})] \right)$$

sum over multiple permutations



GraphSAGE embedding update

- Regardless of node message aggregation, the same update is applied

$$\mathbf{h}_v^{(k+1)} = \sigma \left(\mathbf{W}^{(k)} \left[\mathbf{h}_v^{(k)} \parallel \mathbf{m}_{N(v)}^{(k)} \right] \right)$$

vector concatenation
with node's previous
layer embedding

- (Optional) ℓ_2 normalization at every layer

$$\mathbf{h}_v^{(k)} \leftarrow \frac{\mathbf{h}_v^{(k)}}{\|\mathbf{h}_v^{(k)}\|_2}$$

- Puts all embeddings on the same scale (same ℓ_2 norm)
- Tends to improve performance (though not always)



Graph attention networks

- Idea: weight neighbor nodes differently based on some importance function

$$\mathbf{m}_{N(v)}^{(k)} = \text{Agg}^{(k)} \left(\left\{ \alpha_{vu}^{(k)} \mathbf{h}_u^{(k)}, \forall u \in N(v) \cup v \right\} \right)$$

where $\alpha_{vu}^{(k)}$ is the attention weight placed on $\mathbf{h}_u^{(k)}$

- In GCN / GraphSage
 - $\alpha_{vu}^{(k)} = \frac{1}{|N(v)|} \forall k$ provides equal attention to all neighbor nodes for a given node, v , i.e., all nodes are equally important to v
 - However, governed by graph structure, specifically $\deg(v)$ (node degree), i.e.

$$\alpha_{v_1 u}^{(k)} = \frac{1}{\deg(v_1)} \neq \alpha_{v_2 u}^{(k)} = \frac{1}{\deg(v_2)}$$



Graph attention networks

- Idea: weight neighbor nodes differently based on some importance function

$$\mathbf{m}_{N(v)}^{(k)} = \text{Agg}^{(k)} \left(\left\{ \alpha_{vu}^{(k)} \mathbf{h}_u^{(k)}, \forall u \in N(v) \cup v \right\} \right)$$

where $\alpha_{vu}^{(k)}$ is the attention weight placed on $\mathbf{h}_u^{(k)}$

- What if neighbor nodes are not equally important?
 - Attention, $\alpha_{vu}^{(k)}$, should focus on the important parts of the input data and discard the rest
 - Can we learn a function that selects the important parts of the input?



Graph attention networks

- **Goal:** Learn **attention weights** $\alpha_{vu}^{(k)}$ that place optimal importance on node neighbors for each graph node
- **Idea:** Compute node embeddings, $\mathbf{h}_v^{(k)}$, using an **attention strategy**:
 - Nodes should attend over their neighborhoods' messages
 - Implicit attention weight specification (i.e., formulate this in a learnable manner)



Graph attention networks message aggregation + attention

- Formulate attention in the message passing framework

$$\mathbf{m}_{N(v)}^{(k)} = \sum_{u \in N(v)} \alpha_{v,u}^{(k)} \mathbf{h}_u^{(k)}$$

where $\alpha_{v,u}^{(k)}$ is the attention on neighbor $u \in N(v)$ when aggregating information at node u .

- Many ways to compute $\alpha_{v,u}^{(k)}$, but in the original paper

$$\alpha_{v,u}^{(k)} = \frac{\exp\left(\mathbf{a}^{(k)T} \left[\mathbf{W}^{(k)} \mathbf{h}_v^{(k)} \mid \mathbf{W}^{(k)} \mathbf{h}_u^{(k)} \right]\right)}{\sum_{u' \in N(v)} \exp\left(\mathbf{a}^{(k)T} \left[\mathbf{W}^{(k)} \mathbf{h}_v^{(k)} \mid \mathbf{W}^{(k)} \mathbf{h}_{u'}^{(k)} \right]\right)}$$

scalar attention

$\mathbf{a}^{(k)} \in \mathbb{R}^{2d_{k+1} \times 1}$
is a learnable attention vector

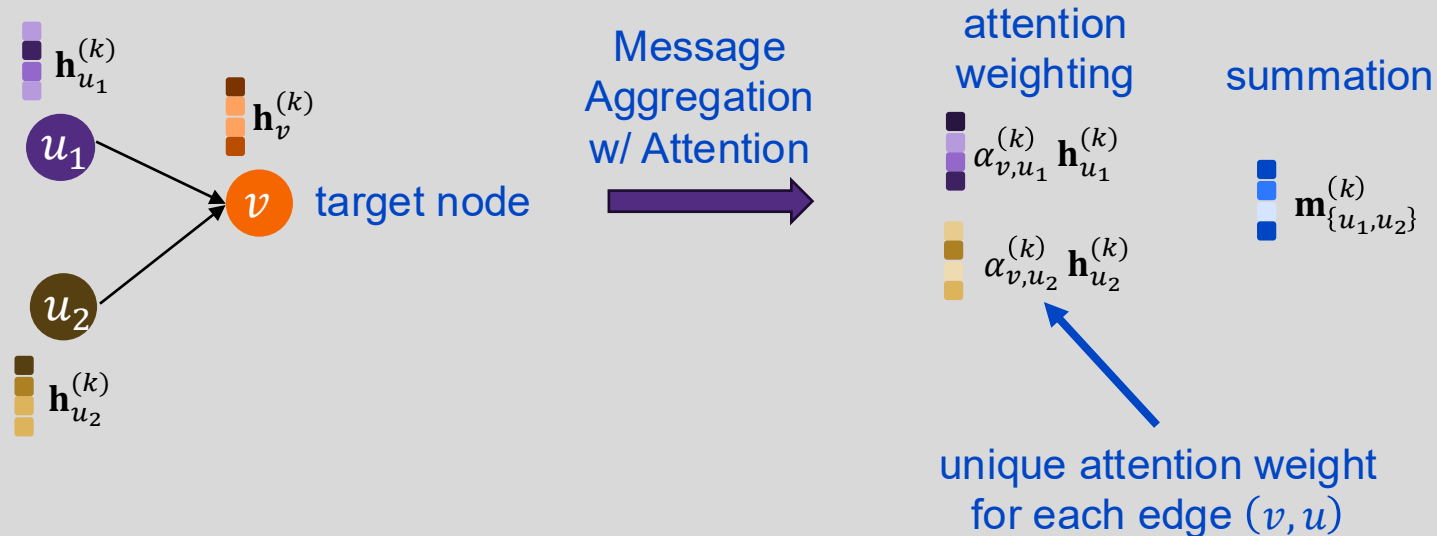
$\mathbf{W}^{(k)} \in \mathbb{R}^{d_{k+1} \times d_k}$
is a learnable weight matrix



Graph attention networks message aggregation + attention

- Formulate attention in the message passing framework

$$\mathbf{m}_{N(v)}^{(k)} = \sum_{u \in N(v)} \alpha_{v,u}^{(k)} \mathbf{h}_u^{(k)}$$





Graph attention message aggregation in matrix form

- How many attention weights, $\alpha_{v,u}^{(k)}$, do we compute at a given layer?
 - $2m$ where m is the number of edges in the graph. Why?
- We can put these in a matrix, $\tilde{\mathbf{W}}^{(k)} \in \mathbb{R}^{|V| \times |V|}$, where

$$\tilde{w}_{v,u}^{(k)} = \begin{cases} \alpha_{v,u}^{(k)} & \text{if } (u, v) \in \mathcal{E} \\ 0 & \text{if } (u, v) \notin \mathcal{E} \end{cases}$$

- Now, we can write the message aggregation in matrix form as

$$\mathbf{M}^{(k)} = \tilde{\mathbf{W}}^{(k)} \mathbf{H}^{(k)}$$

where $\mathbf{H}^{(k)} \in \mathbb{R}^{|V| \times d_k}$ so that $\mathbf{M}^{(k)} \in \mathbb{R}^{|V| \times d_k}$

This formulation assumes the attention weights have already been computed.



Graph attention with convolution update

- Assuming we have the message aggregation in matrix

$$\mathbf{M}^{(k)} = \tilde{\mathbf{W}}^{(k)} \mathbf{H}^{(k)} \in \mathbb{R}^{|V| \times d_k}$$

we update the node embeddings via any differentiable update function

$$\mathbf{H}^{(k+1)} = \text{Update}^{(k)}(\mathbf{H}^{(k+1)}, \mathbf{M}^{(k)})$$

- The typical update function applies the same approach as graph convolution networks

$$\mathbf{H}^{(k+1)} = \sigma \left(\mathbf{W}^{(k)T} \in \mathbb{R}^{|V| \times d_{k+1}} \right)$$

This is the same $\mathbf{W}^{(k)}$ used in the attention weight computations

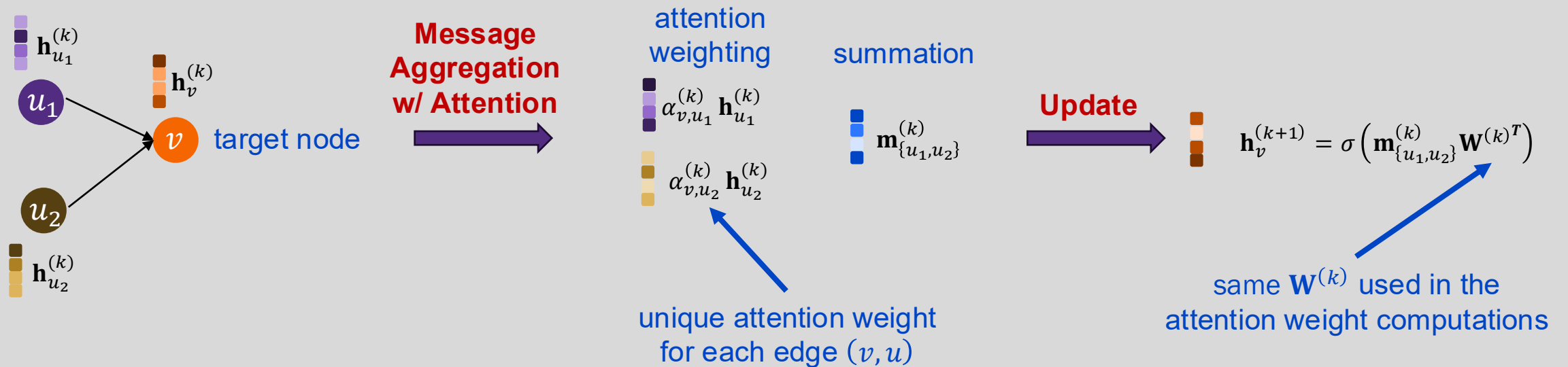


Graph attention with convolution update

- Formulate attention in the message passing framework

$$\mathbf{m}_{N(v)}^{(k)} = \sum_{u \in N(v)} \alpha_{v,u}^{(k)} \mathbf{h}_u^{(k)}$$

$$\mathbf{h}^{(k+1)} = \sigma \left(\mathbf{m}_{N(v)}^{(k)} \mathbf{W}^{(k)T} \right)$$





Graph attention with convolution update layer

Putting it all together, **graph attention with convolution** update involves the **following steps** at each layer:

1. Compute the attention weights

$$\alpha_{v,u}^{(k)} = \frac{\exp \left(\mathbf{a}^{(k)T} \left[\mathbf{W}^{(k)} \mathbf{h}_v^{(k)} \mid \mathbf{W}^{(k)} \mathbf{h}_u^{(k)} \right] \right)}{\sum_{u' \in N(v)} \exp \left(\mathbf{a}^{(k)T} \left[\mathbf{W}^{(k)} \mathbf{h}_v^{(k)} \mid \mathbf{W}^{(k)} \mathbf{h}_{u'}^{(k)} \right] \right)}$$

2. Aggregate node messages with attention weighting

$$\mathbf{m}_{N(v)}^{(k)} = \sum_{u \in N(v)} \alpha_{v,u}^{(k)} \mathbf{h}_u^{(k)} \rightarrow \mathbf{M}^{(k)} = \tilde{\mathbf{W}}^{(k)} \mathbf{H}^{(k)}$$

3. Update node embeddings

$$\mathbf{h}_v^{(k+1)} = \mathbf{m}_{N(v)}^{(k)} \mathbf{W}^{(k)T} \rightarrow \mathbf{H}^{(k+1)} = \mathbf{M}^{(k)} \mathbf{W}^{(k)T} \in \mathbb{R}^{|V| \times d_{k+1}}$$



Multi-head graph attention

- Use H different *attention heads* to aggregate messages
- Results in H different messages, one for each attention head, $h \in [1, H]$
- Stabilizes attention mechanism learning

$$\mathbf{m}_{N(v)}^{(k,h)} = \sum_{u \in N(v)} \alpha_{v,u}^{(k,h)} \mathbf{h}_u^{(k)}$$

where

$$\alpha_{v,u}^{(k)} = \frac{\exp \left(\mathbf{a}^{(k,h)T} \left[\mathbf{W}^{(k,h)} \mathbf{h}_v^{(k)} \mid \mathbf{W}^{(k,h)} \mathbf{h}_u^{(k)} \right] \right)}{\sum_{u' \in N(v)} \exp \left(\mathbf{a}^{(k,h)T} \left[\mathbf{W}^{(k,h)} \mathbf{h}_v^{(k)} \mid \mathbf{W}^{(k,h)} \mathbf{h}_{u'}^{(k)} \right] \right)}$$

$$\mathbf{a}^{(k,h)} \in \mathbb{R}^{2d_{k+1} \times 1}$$

different attention vector for
each attention head

$$\mathbf{W}^{(k,h)} \in \mathbb{R}^{d_{k+1} \times d_k}$$

different weight matrix for
each attention head



Multi-head graph attention with convolution update

- Aggregated message from each attention head is updated individually
- Updates are aggregated (e.g., concatenation or summation) to form new embedding for node v

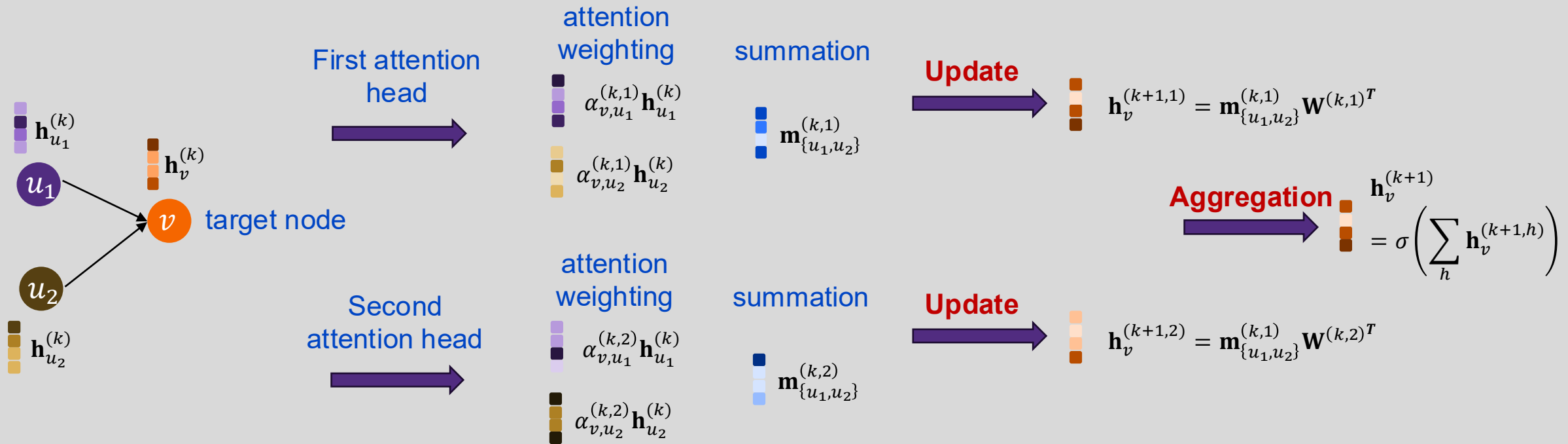
$$\mathbf{m}_{N(v)}^{(k,h)} = \sum_{u \in N(v)} \alpha_{v,u}^{(k,h)} \mathbf{h}_u^{(k)}$$

$$\mathbf{h}^{(k+1,h)} = \mathbf{m}_{N(v)}^{(k,h)} \mathbf{W}^{(k,h)T}$$

$$\mathbf{h}^{(k+1)} = \sigma \left(\sum_h \mathbf{h}^{(k+1,h)} \right)$$

Multi-head graph attention with convolution update

- Here, assume we have two attention heads





Benefits of Attention

- **Key benefit:** Allows for implicitly specifying different importance values for each neighbor node
- Computational efficiency
 - Attention weights can be computed in parallel across all graph edges
 - Aggregation can be parallelized across all nodes
- Storage efficient
 - Sparse matrix operations require $\leq O(V + E)$ storage
 - Fixed number of model parameters independent of graph size (node count)
- A note about **implementation:**
 - The message passing framework is useful for conceptual understanding of GNNs
 - In practice, many of the matrix operations are implemented in ways that mix aspects of neighbor node aggregation and node update for efficiency



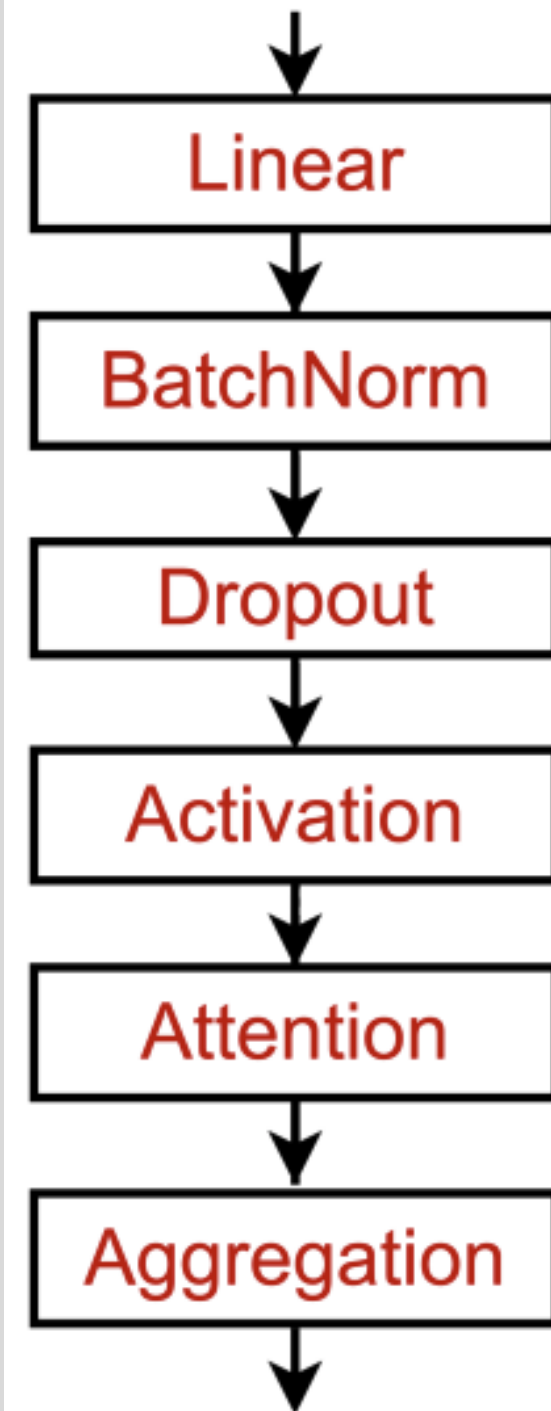
Applicable deep learning tools





A GNN layer in practice

- We often want to use familiar deep learning modules to create our GNN layers
 - Batch normalization – stabilize network training
 - Dropout – prevent overfitting
 - Many other useful deep learning modules





Batch normalization

- Goal: stabilize network training
- Approach: Given a batch of N inputs (e.g., node embeddings)
 - Recenter the embeddings to zero mean
 - Rescale the embeddings to unit variance

Input: $\mathbf{X} \in \mathbb{R}^{N \times d}$

Learnable Parameters:
 $\gamma, \beta \in \mathbb{R}^d$

Output: $\mathbf{Y} \in \mathbb{R}^{N \times d}$

Step 1:

Compute the mean and variance for each latent feature (embedding column j) over the N embeddings

$$u_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$
$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

Step 2:

Normalize the embedding feature

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$
$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$



Dropout regularization in standard neural networks

- Regularize a deep neural network to prevent overfitting
- Approach
 - During training: with some probability p , randomly turn off network neurons (i.e., set output to zero)
 - During test / deployment: use all neurons

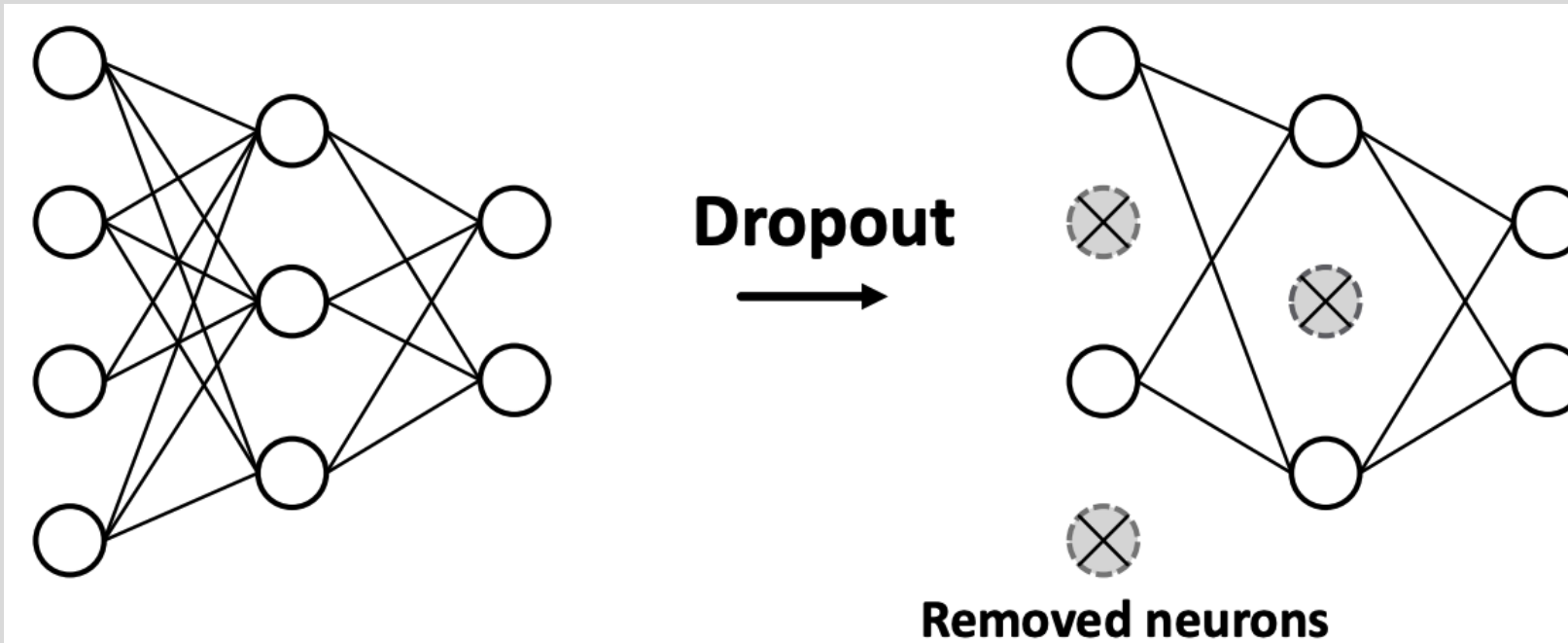
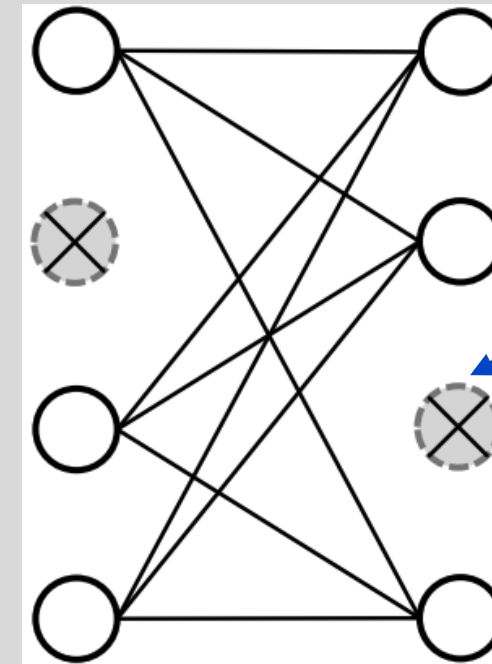
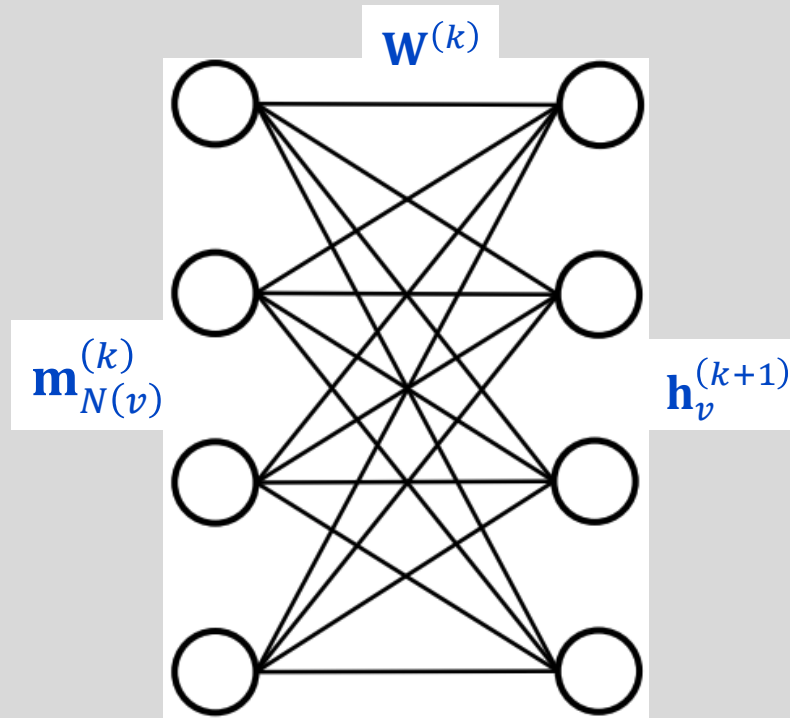


Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



Dropout regularization in GNNs

- In a GNN, dropout is applied in the *update* function
- Example, in GCN the update for node v is $\mathbf{h}_v^{(k+1)} = \sigma \left(\mathbf{W}^{(k)} \mathbf{m}_{N(v)}^{(k)} \right)$
 - Dropout implemented by removing neurons in $\mathbf{W}^{(k)}$

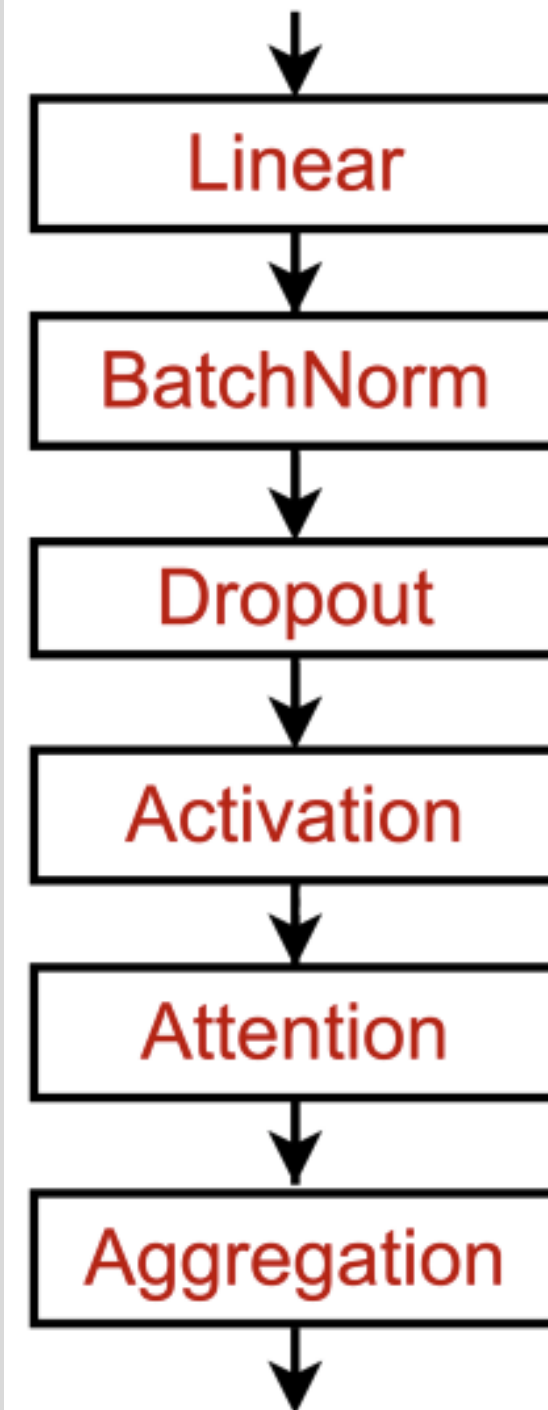


Dropout removes latent features from the aggregated message, **NOT** nodes in the graph



GNN layers in practice

- Modern deep learning modules are useful in developing practical GNN layers
- **Designing novel GNN layers is an active research domain**





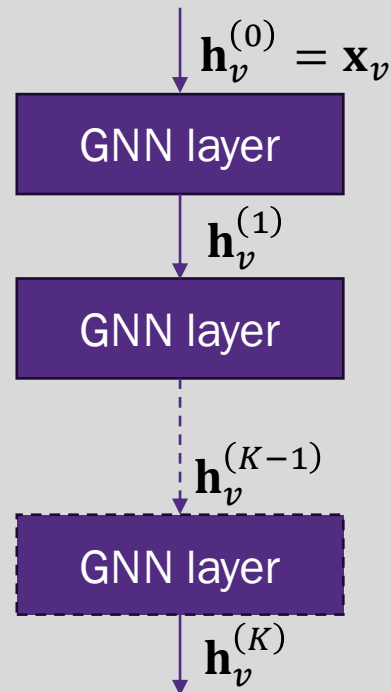
Stacking GNN layers





Stacking GNN layers

- How do we construct a GNN with multiple layers?
- Standard approach is to stack layers sequentially
- Initial input: $\mathbf{h}_v^{(0)} = \mathbf{x}_v$ is the **input feature vector** for v
- Output: Node embedding $\mathbf{h}_v^{(K)}$ after K layers





Over-smoothing in deep GNNs

- Stacking many GNN layers can cause over-smoothing
- Over-smoothing occurs when all node embeddings converge to the same value – i.e., all embedding vectors are the same
- **Why does this happen?**



Receptive field of a GNN

- Receptive field: the set of nodes that contribute to the aggregated neighborhood message for a node of interest
- In a K -layer GNN, each node has a receptive field of its K -hop neighborhood

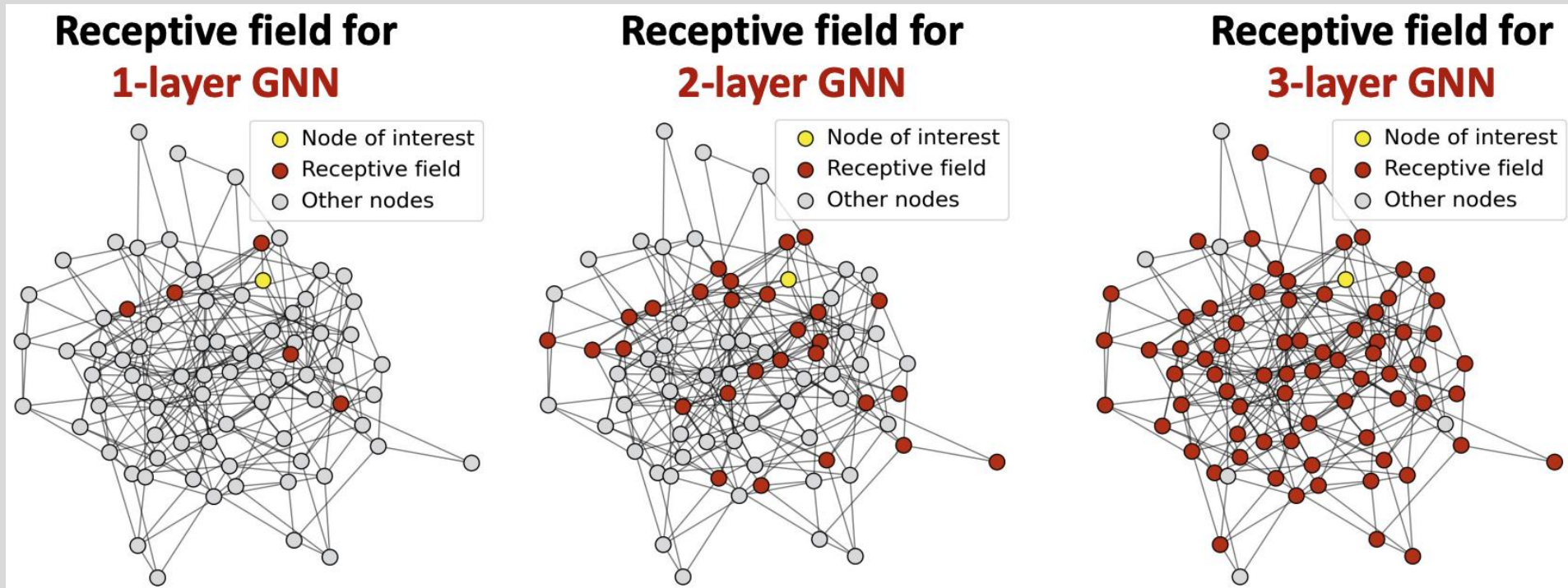


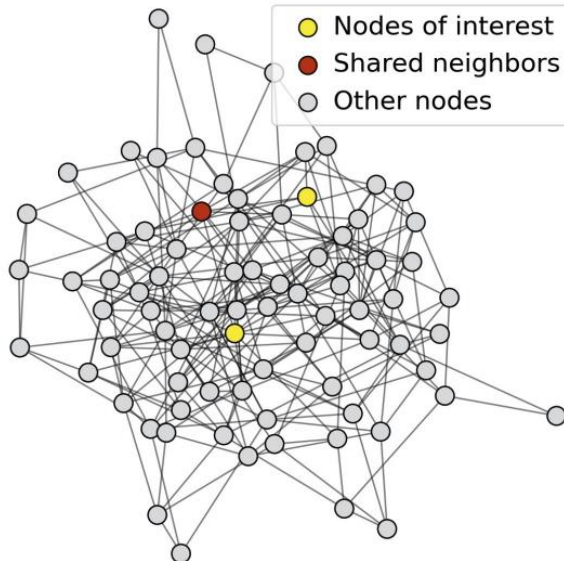
Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



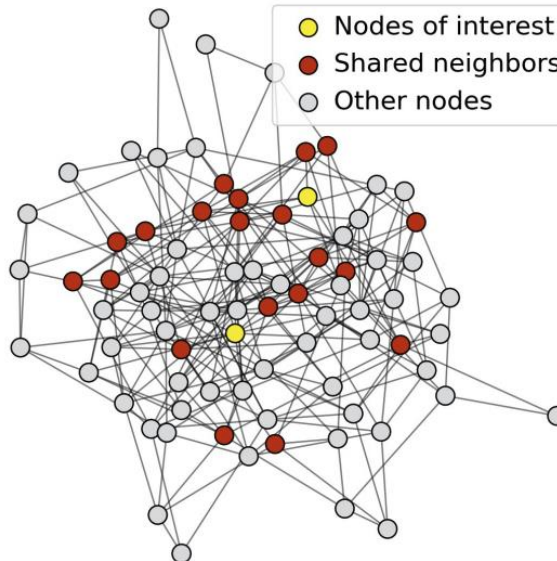
Receptive field of a GNN

- Receptive field overlap for two nodes
- The shared neighbors quickly grows when increasing the field hop size (i.e., number of GNN layers)
- Results in **very similar messages** being input to each node's update function, yielding similar embeddings for both

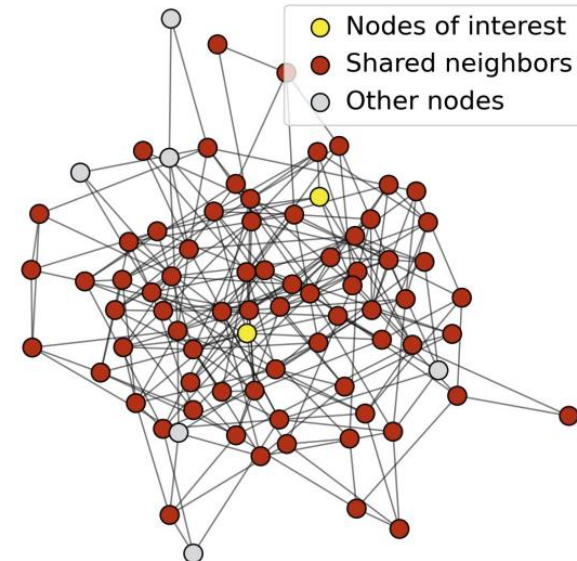
1-hop neighbor overlap
Only 1 node



2-hop neighbor overlap
About 20 nodes



3-hop neighbor overlap
Almost all the nodes!





Addressing over-smoothing

- Be conservative with adding GNN layers – adding too many layers will cause over-smoothing
- Analyze the receptive field necessary for your problem (e.g., by computing the network diameter – longest shortest path)
- Set the number of GNN layers to be slightly more (say plus 1-3) than the estimated receptive field requirement

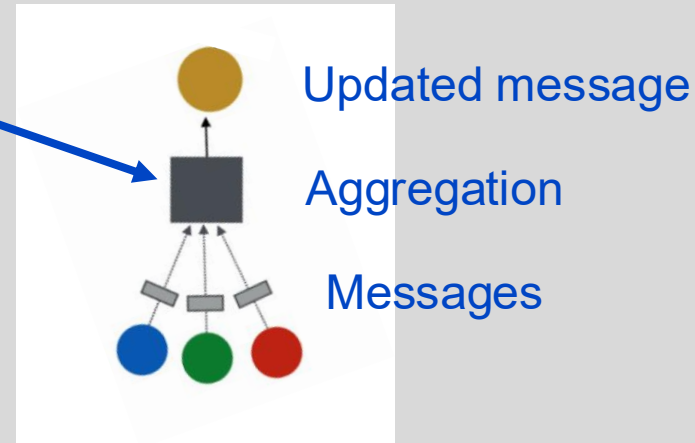
How can we enhance the expressive power of a GNN if the receptive field requirement limits the model to a small number of GNN layers?



Increasing expressive power for GNNs

- Increase the expressive power within the GNN layer
- In the GNN layers we discussed (GCN, GraphSAGE, GAT), the aggregations and transformations only include one linear layer
- We can add layers to these to form a deeper network within the layer – this does not change the receptive field

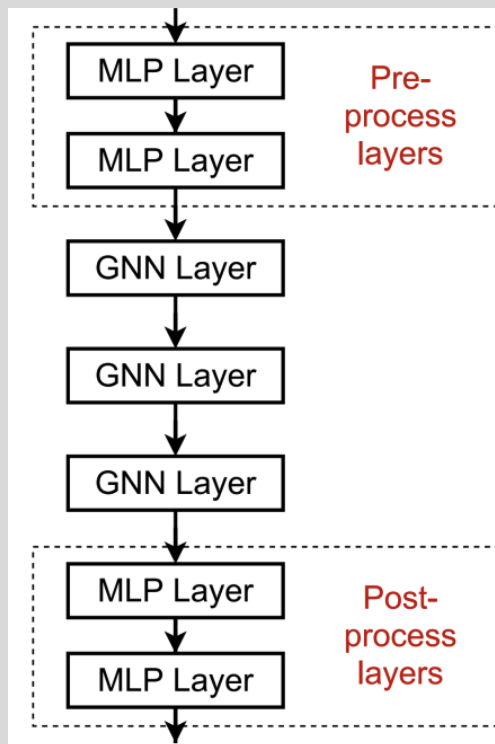
The aggregation and updates can use an MLP





Increasing expressive power for GNNs

- Add network layers that do not pass messages
- The overall network can contain non-message passing (GNN) layers
 - Example: add MLP layers before and after GNN layers to pre/post process next/prior layer output



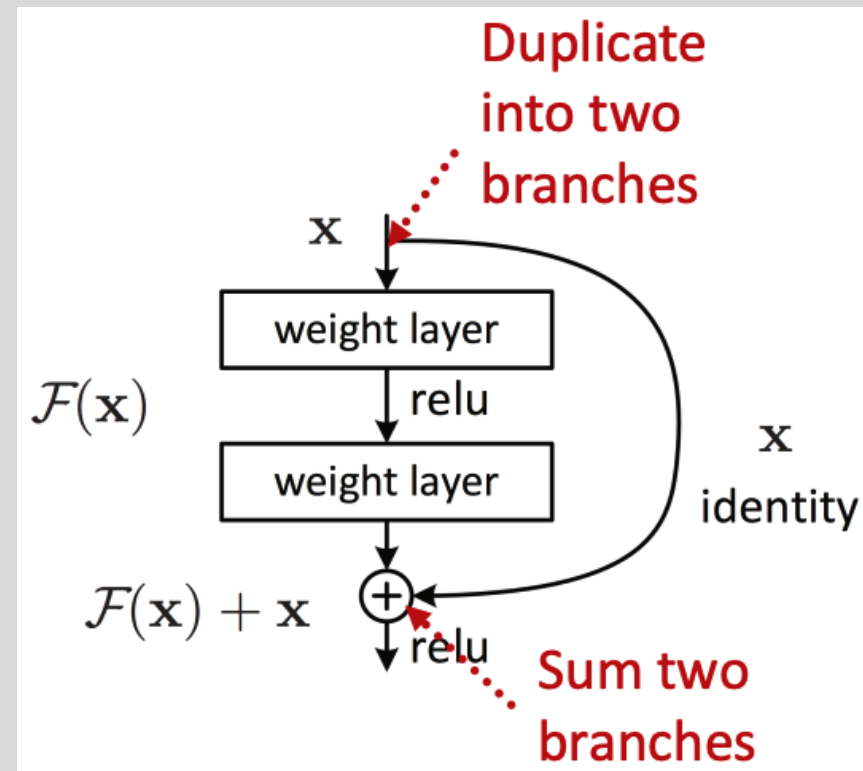
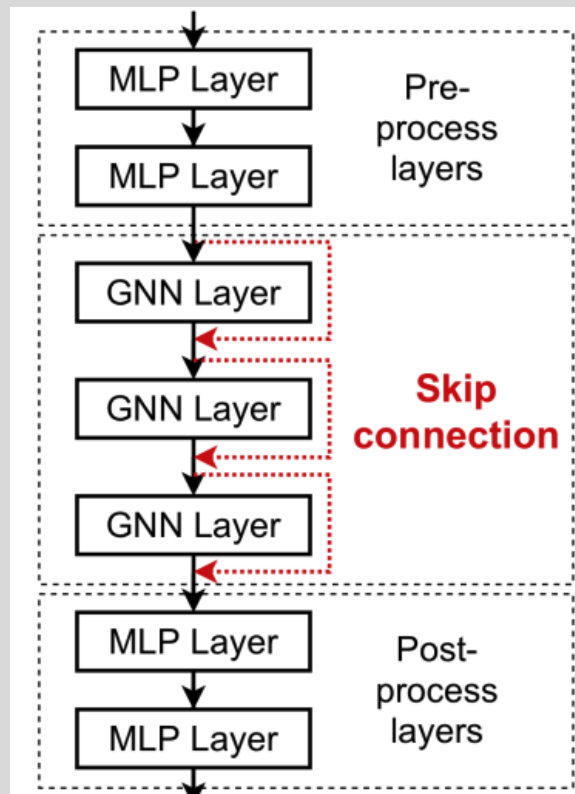
Pre-processing: important when encoding features is necessary (e.g., when nodes represent images or text)

Post-processing: important when reasoning / transformation over node embeddings is required (e.g., graph classification)



But I really need more GNN layers! Skip connections

- Skip connections can be used to directly connect earlier GNN layers to deeper layers
- The node embeddings from earlier layers may prevent over-smoothing
- Acts like a mixture of shallow GNNs and deep GNNs





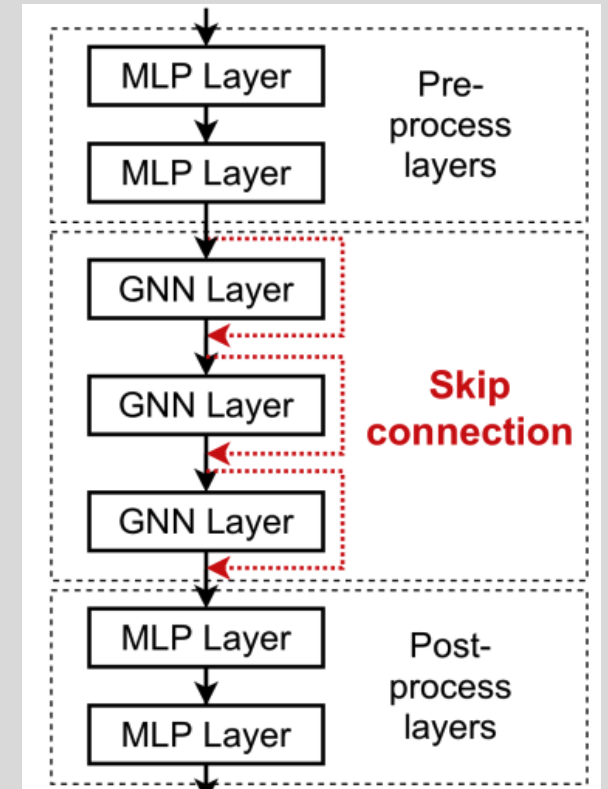
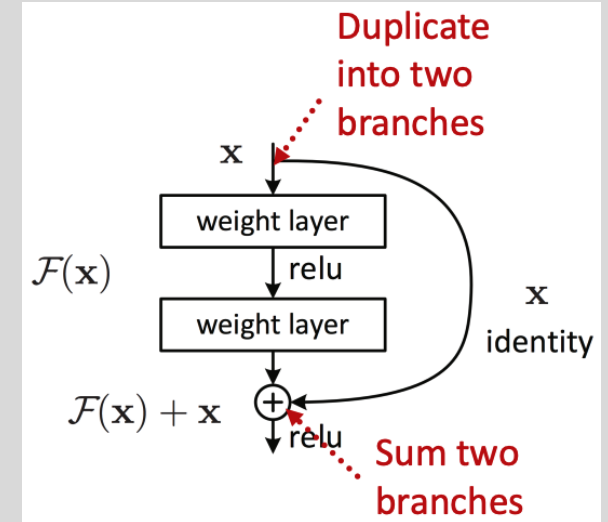
GCN with skip connections

- Standard GCN layer

$$\mathbf{h}_v^{(k+1)} = \sigma \left(\underbrace{\mathbf{w}^{(k)} \sum_{u \in N(v) \cup \{v\}} \frac{\mathbf{h}_u^{(k)}}{|N(v)|}}_{f(x)} \right)$$

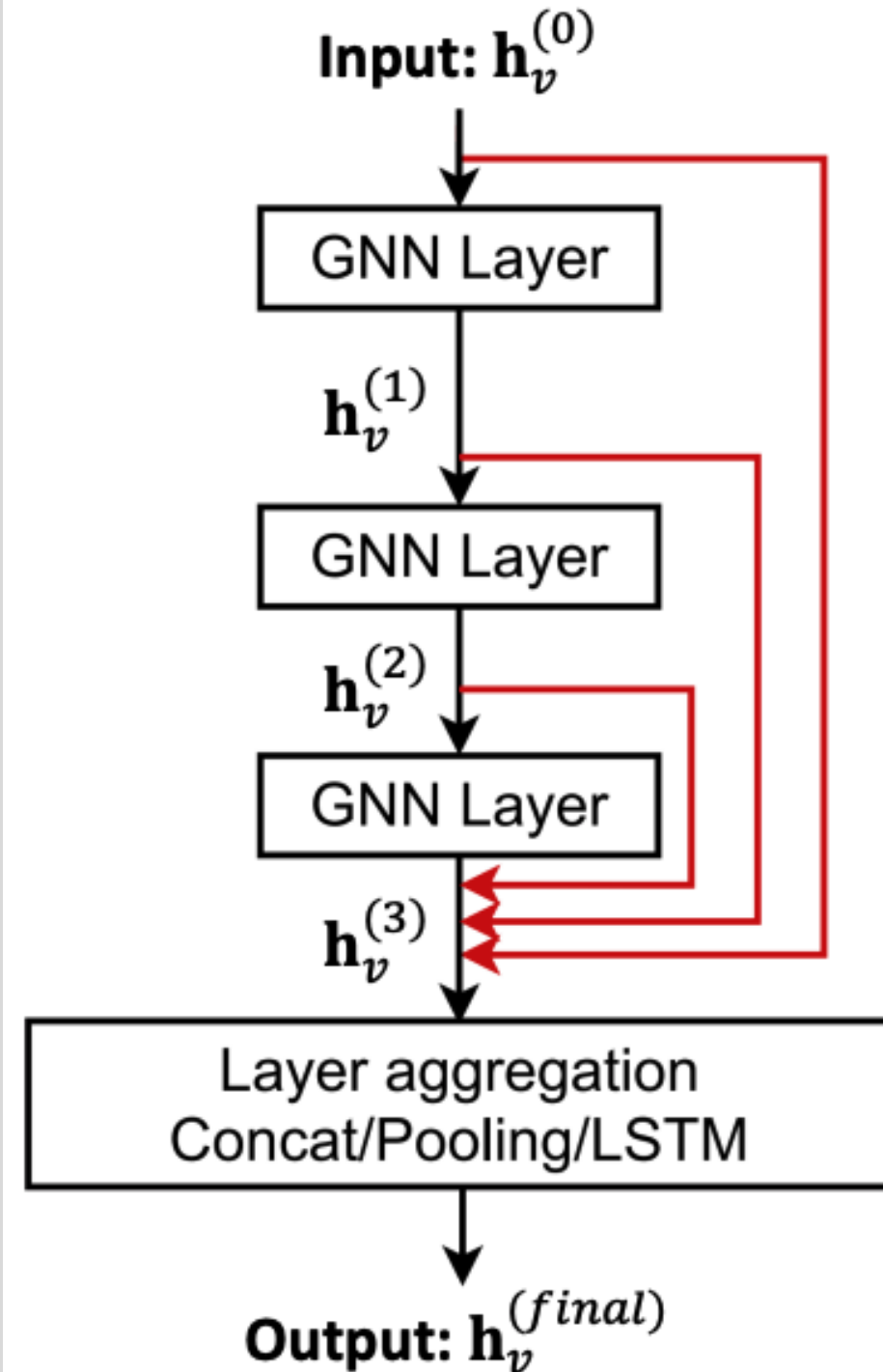
- GCN layer with skip connection from layer $k - j$ (typically $j = 1$)

$$\mathbf{h}_v^{(k+1)} = \sigma \left(\underbrace{\mathbf{h}_v^{(k-j)}}_x + \underbrace{\mathbf{w}^{(k)} \sum_{u \in N(v) \cup \{v\}} \frac{\mathbf{h}_u^{(k)}}{|N(v)|}}_{f(x)} \right)$$



Other skip connection options

- Directly skip to the last layer
- The final layer aggregates from all the GNN layer node embeddings in the previous layers





Graph manipulations





Why manipulate graphs

- We've assumed that the network graph = the GNN structure
- We might want to break this assumption if:
 - The input graph lacks features → no messages
 - The graph is too sparse → inefficient message passing
 - The graph is too dense → message passing computation is expensive
 - The graph is too large → cannot fit the computation graph into a GPU



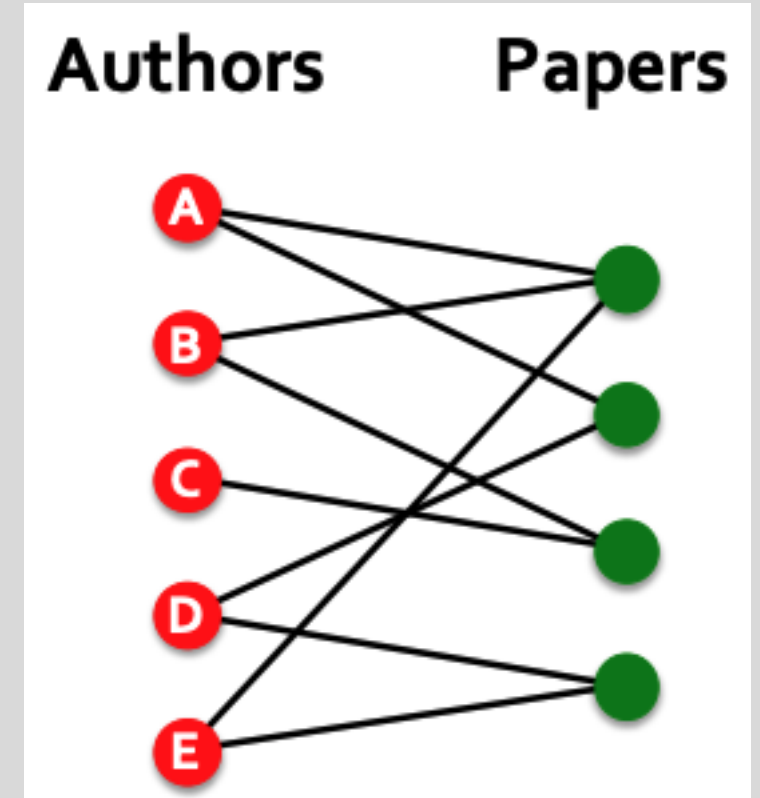
Why manipulate graphs

- We've assumed that the network graph = the GNN structure
- We might want to break this assumption if:
 - The input graph lacks features → apply feature augmentation (later lecture)
 - The graph is too sparse → add virtual nodes
 - The graph is too dense → apply GraphSAGE or other neighbor sampling
 - The graph is too large → sample subgraphs to compute embeddings (later lecture, time permitting)



Adding virtual edges

- Goal is to augment a sparse graph
- Common approach is to connect 2-hop neighbors via virtual edges
- Mathematically, equivalent to using $\mathbf{A} + \mathbf{A}^2$ instead of the adjacency matrix
- Common use case is bipartite graphs
 - Author-to-papers (they authored)
 - 2-hop virtual edges make an author-author collaboration graph





Adding virtual nodes

- Goal is to augment a sparse graph
- Virtual node will connect to all nodes in the graph (or subgraph)
 - Suppose in a sparse graph, two nodes have a shortest path distance > 2
 - After adding the virtual node, all nodes have a shortest path distance of 2
 - node U – virtual node – node V
 - Greatly improves message passing

The virtual node

